

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/gtopt.py ==  
['C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/gtopt.py']
```

Errors and Exceptions:

Python Errors and Built-in Exceptions: Python (interpreter) raises exceptions when it encounters **errors**. When writing a program, we, more often than not, will encounter errors. Error caused by not following the proper structure (syntax) of the language is called syntax error or parsing error

ZeroDivisionError:

ZeroDivisionError in Python indicates that the second argument used in a division (or modulo) operation was zero.

OverflowError:

OverflowError in Python indicates that an arithmetic operation has exceeded the limits of the current Python runtime. This is typically due to excessively large float values, as integer values that are too big will opt to raise memory errors instead.

ImportError:

It is raised when you try to import a module which does not exist. This may happen if you made a typing mistake in the module name or the module doesn't exist in its standard path. In the example below, a module named "non_existing_module" is being imported but it doesn't exist, hence an import error exception is raised.

IndexError:

An IndexError exception is raised when you refer a sequence which is out of range. In the example below, the list abc contains only 3 entries, but the 4th index is being accessed, which will result an IndexError exception.

TypeError:

When two unrelated type of objects are combined, TypeErrorexception is raised. In example below, an int and a string is added, which will result in TypeError exception.

IndentationError:

Unexpected indent. As mentioned in the "expected an indented block" section, Python not only insists on indentation, it insists on consistent indentation. You are free to choose the number of spaces of indentation to use, but you then need to stick with it.

Syntax errors:

These are the most basic type of error. They arise when the Python parser is unable to understand a line of code. Syntax errors are almost always fatal, i.e. there is almost never a way to successfully execute a piece of code containing syntax errors.

Run-time error:

A run-time error happens when Python understands what you are saying, but runs into trouble when following your instructions.

Key Error :

Python raises a `KeyError` whenever a `dict()` object is requested (using the format `a = adict[key]`) and the key is not in the dictionary.

Value Error:

In Python, a value is the information that is stored within a certain object. To encounter a `ValueError` in Python means that is a problem with the content of the object you tried to assign the value to.

Python has many built-in exceptions which forces your program to output an error when something in it goes wrong. In Python, users can define such exceptions by creating a new class. This exception class has to be derived, either directly or indirectly, from `Exception` class.

Different types of exceptions:

- `ArrayIndexOutOfBoundsException`.
- `ClassNotFoundException`.
- `FileNotFoundException`.
- `IOException`.
- `InterruptedException`.
- `NoSuchFieldException`.
- `NoSuchMethodException`

Handling Exceptions:

The cause of an exception is often external to the program itself. For example, an incorrect input, a malfunctioning IO device etc. Because the program abruptly terminates on encountering an exception, it may cause damage to system resources, such as files. Hence, the exceptions should be properly handled so that an abrupt termination of the program is prevented.

Python uses try and except keywords to handle exceptions. Both keywords are followed by indented blocks.

Syntax:

try :

 #statements in try block

except :

 #executed when error in try block

Typically we see, most of the times

- **Syntactical errors** (wrong spelling, colon (:) missing),
At developer level and compile level it gives errors.
- **Logical errors** (2+2=4, instead if we get output as 3 i.e., wrong output),
As a developer we test the application, during that time logical error may obtained.
- **Run time error** (In this case, if the user doesn't know to give input, 5/6 is ok but if the user say 6 and 0 i.e., 6/0 (shows error a number cannot be divided by zero))
This is not easy compared to the above two errors because it is not done by the system, it is (mistake) done by the user.

The things we need to observe are:

1. You should be able to understand the mistakes; the error might be done by user, DB connection or server.
2. Whenever there is an error execution should not stop.
Ex: Banking Transaction
3. The aim is execution should not stop even though an error occurs.

For ex:

```
a=5
```

```
b=2
```

```
print(a/b)
```

```
print("Bye")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex1.py
```

```
2.5
```

```
Bye
```

- The above is normal execution with no error, but if we say when $b=0$, it is a critical and gives error, see below

```
a=5
```

```
b=0
```

```
print(a/b)
```

```
print("bye") #this has to be printed, but abnormal termination
```

Output:

Traceback (most recent call last):

File "C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex2.py", line 3, in <module>

```
print(a/b)
```

ZeroDivisionError: division by zero

- To overcome this we handle exceptions using except keyword

```
a=5
```

```
b=0
```

```
try:  
    print(a/b)  
except Exception:  
    print("number can not be divided by zero")  
    print("bye")
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex3.py

number can not be divided by zero

bye

- **The except block executes only when try block has an error, check it below**

a=5

b=2

```
try:
```

```
    print(a/b)
```

```
except Exception:
```

```
    print("number can not be divided by zero")
```

```
    print("bye")
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex4.py

2.5

- **For example if you want to print the message like what is an error in a program then we use “e” which is the representation or object of an exception.**

a=5

b=0

```
try:
```

```
print(a/b)

except Exception as e:

    print("number can not be divided by zero",e)

print("bye")
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex5.py

number can not be divided by zero **division by zero**

bye



(Type of error)

Let us see some more examples:

I don't want to print bye but I want to close the file whenever it is opened.

```
a=5
```

```
b=2
```

```
try:
```

```
    print("resource opened")
```

```
    print(a/b)
```

```
    print("resource closed")
```

```
except Exception as e:
```

```
    print("number can not be divided by zero",e)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex6.py

resource opened

2.5

resource closed

- **Note:** the file is opened and closed well, but see by changing the value of b to 0,

```
a=5
```

```
b=0
```

```
try:
```

```
    print("resource opened")
```

```
    print(a/b)
```

```
    print("resource closed")
```

```
except Exception as e:
```

```
    print("number can not be divided by zero",e)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex7.py
```

```
resource opened
```

```
number can not be divided by zero division by zero
```

- **Note:** resource not closed
- **To overcome this, keep print("resource closed") in except block, see it**

```
a=5
```

```
b=0
```

```
try:
```

```
    print("resource opened")
```

```
    print(a/b)
```

```
except Exception as e:
```

```
    print("number can not be divided by zero",e)
```

```
    print("resource closed")
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex8.py

resource opened

number can not be divided by zero division by zero

resource closed

- **The result is fine that the file is opened and closed, but again change the value of b to back (i.e., value 2 or other than zero)**

```
a=5
```

```
b=2
```

```
try:
```

```
    print("resource opened")
```

```
    print(a/b)
```

```
except Exception as e:
```

```
    print("number can not be divided by zero",e)
```

```
    print("resource closed")
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex9.py

resource opened

2.5

- **But again the same problem file/resource is not closed**
- **To overcome this python has a feature called **finally:****



This block gets executed though we get an error or not

Note: **Except block executes, only when **try** block has an error, but **finally** block executes, even though you get an exception.**

```
a=5
```

```
b=0
```

```
try:
```



```
print("resource open")
print(a/b)
k=int(input("enter a number"))
print(k)
except ZeroDivisionError as e:
    print("the value can not be divided by zero",e)
finally:
    print("resource closed")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex10.py
resource open
the value can not be divided by zero division by zero
resource closed
```

- **change the value of b to 2 for above program, you see the output like**

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex10.py
resource open
2.5
enter a number 6
6
resource closed
```

- **Instead give input as some character or string for above program, check the output**

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex10.py
resource open
2.5
enter a number p
resource closed
```

Traceback (most recent call last):

File "C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex10.py", line 7, in <module>

```
k=int(input("enter a number"))
```

ValueError: invalid literal for int() with base 10: ' p'